ESPRIT
École Supérieure Privée d'Ingénierie et de Technologies

APPLICATION WEB DISTRIBUÉE

SAKANI

Plateforme intelligente de logement étudiant

Find your place. Find your people.

SAKANI est une plateforme web conçue pour simplifier la recherche de logements étudiants et faciliter la colocation grâce à un système intelligent de matching, tout en centralisant la gestion des annonces, des candidatures et des visites.

Microservices | Spring Boot | Keycloak

ÉQUIPE DE RÉALISATION

Mayar Ben Ghars
Mayssa Tabassi
Souhir Meddeb
Rane Jaffel

Introduction générale

Contexte, problématique et objectifs du projet

Le logement représente l'un des besoins essentiels pour les étudiants, particulièrement pour ceux qui doivent poursuivre leurs études dans une ville éloignée de leur domicile familial. En Tunisie, la recherche d'un logement étudiant constitue souvent un véritable défi, notamment en raison des contraintes budgétaires, de la dispersion des annonces immobilières et de la difficulté à trouver des colocataires compatibles.

Face à ces difficultés, les étudiants sont amenés à consulter différentes plateformes et à multiplier les échanges avec les propriétaires afin de trouver un logement correspondant à leurs besoins. Ce processus peut devenir long et complexe, notamment lorsqu'il s'agit de comparer les offres, de déposer des candidatures, d'organiser des visites ou de trouver des personnes avec lesquelles partager un logement.

C'est dans ce contexte que s'inscrit notre projet **SAKANI**, une plateforme web dédiée au logement étudiant. Son objectif principal est de centraliser les différentes étapes de la recherche de logement et de simplifier les interactions entre les étudiants et les propriétaires.

La plateforme permet aux propriétaires de publier et de gérer leurs annonces immobilières, tandis que les étudiants peuvent rechercher des logements selon plusieurs critères, consulter les informations disponibles, déposer des candidatures et organiser des visites. Elle intègre également un système intelligent de matching permettant de proposer des colocataires potentiellement compatibles selon leurs préférences et leurs habitudes de vie.

1. Problématique

Comment faciliter l'accès à un logement étudiant adapté, abordable et sécurisé, tout en simplifiant la recherche de colocataires compatibles ?

2. Objectifs du projet

Le développement de SAKANI vise à répondre à cette problématique à travers plusieurs objectifs :

- Centraliser les annonces de logements étudiants au sein d'une plateforme unique.
- Faciliter la recherche de logements grâce à des critères de filtrage adaptés.
- Simplifier la gestion et le suivi des candidatures des étudiants.
- Organiser les visites en facilitant la coordination entre étudiants et propriétaires.
- Proposer un système de matching pour identifier des colocataires compatibles.

— Offrir une plateforme modulaire, sécurisée et évolutive reposant sur une architecture en microservices.

3. Approche technique

Sur le plan technique, l'application repose sur une architecture distribuée en microservices. Le backend est développé avec Spring Boot et Spring Cloud. L'architecture intègre Eureka pour la découverte des services, Spring Cloud Gateway comme point d'entrée unique et Keycloak pour l'authentification et la gestion des rôles.
Chaque microservice possède une responsabilité spécifique et une base de données indépendante. L'ensemble est complété par une interface web permettant aux différents acteurs d'accéder aux fonctionnalités de la plateforme.

4. Organisation de la documentation

Cette documentation présente les différentes étapes de conception et de réalisation du projet SAKANI. Elle aborde l'analyse des besoins, l'architecture globale, la conception des microservices, leurs fonctionnalités, les mécanismes de communication et les aspects techniques de l'implémentation.

Présentation générale du projet

Une solution numérique dédiée au logement étudiant

1. Solution proposée

Pour répondre aux difficultés rencontrées par les étudiants dans leur recherche de logement, SAKANI propose un environnement numérique centralisé permettant de rechercher, comparer et gérer les différentes démarches liées au logement étudiant.

La plateforme met en relation les étudiants et les propriétaires à travers un ensemble de fonctionnalités complémentaires. Elle facilite la publication des annonces, la recherche de logements, la gestion des candidatures et l'organisation des visites.

En complément, SAKANI intègre un système de matching intelligent destiné à identifier des colocataires compatibles en fonction de leurs préférences et de leurs habitudes de vie.

2. Acteurs du système

La plateforme SAKANI repose sur trois acteurs principaux, chacun disposant de responsabilités et de fonctionnalités spécifiques.

01 Étudiant

L'étudiant utilise la plateforme pour rechercher un logement adapté à ses besoins et à son budget.

- Rechercher et consulter les annonces.
- Déposer et suivre des candidatures.
- Consulter et réserver des visites.
- Trouver des colocataires compatibles.

02 Propriétaire

Le propriétaire utilise SAKANI pour publier ses logements et gérer les demandes des étudiants.

- Créer et gérer les annonces immobilières.
- Publier les offres de logement.
- Consulter les candidatures reçues.
- Accepter ou refuser les demandes.
- Proposer et gérer les créneaux de visite.

03 Administrateur

L'administrateur assure la supervision générale et la modération de la plateforme.

- Superviser le fonctionnement de la plateforme.
- Assurer la modération des contenus.
- Veiller au bon déroulement des activités.

3. Fonctionnalités principales

Les principales fonctionnalités de SAKANI sont organisées autour de cinq axes complémentaires :

Recherche intelligente
Recherche multicritère selon la localisation, le budget, les équipements et les places disponibles.

Gestion des candidatures
Soumission et suivi des candidatures avec gestion de leurs différents statuts.

Planification des visites
Organisation et gestion des visites des logements proposés.

Matching des colocataires
Suggestions de colocataires selon les préférences et les habitudes de vie.

Tableau de bord personnalisé
Interface permettant de suivre les logements, les candidatures et les visites.

Architecture globale de SAKANI

Une architecture distribuée composée de cinq microservices

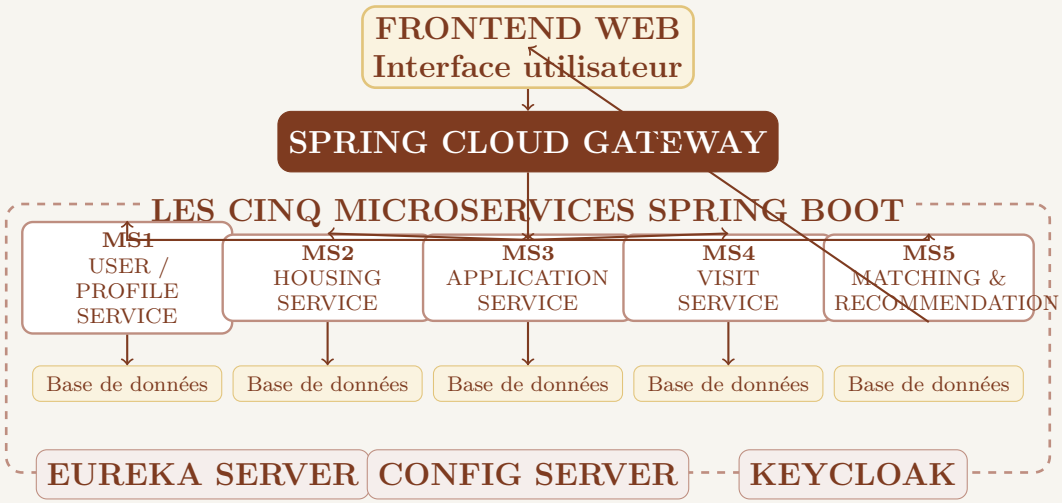
1. Présentation de l'architecture

L'application SAKANI adopte une architecture distribuée basée sur les microservices, développée à l'aide de Spring Boot et Spring Cloud. Cette architecture repose sur la séparation des fonctionnalités en cinq services indépendants, chacun responsable d'un domaine fonctionnel précis.

Cette organisation permet de structurer l'application en modules spécialisés, de faciliter leur maintenance et de favoriser leur évolution indépendante.

Les différents microservices communiquent à travers des API REST, tandis que plusieurs composants d'infrastructure assurent la gestion des requêtes, la découverte des services, la configuration centralisée et la sécurité.

2. Schéma de l'architecture



Vue conceptuelle des composants de l'architecture SAKANI.

3. Découpage fonctionnel des microservices

MS1 – User / Profile Service

Gère les profils des utilisateurs, leurs rôles, leurs préférences de logement et leurs habitudes de vie.

MS2 – Housing Service

Gère les logements, les annonces immobilières, les équipements, les photos et la recherche multicritère.

MS3 – Application Service

Gère le processus de candidature : dépôt, consultation, suivi des statuts, acceptation, refus et annulation des demandes.

MS4 – Visit Service

Gère les créneaux de visite, les demandes de réservation, les confirmations et le suivi des visites.

MS5 – Matching & Recommendation Service

Calcule les scores de compatibilité et propose des logements ainsi que des colocataires adaptés aux préférences des étudiants.

4. Composants d'infrastructure

— **Eureka Server :** enregistrement et découverte dynamique des microservices.
— **Config Server :** centralisation des configurations.
— **Keycloak :** authentification et gestion des rôles.
— **Docker Compose :** orchestration et déploiement des conteneurs.
— **Bases de données :** H2, MySQL et PostgreSQL.

Diagramme global de cas d'utilisation

Vue d'ensemble des interactions entre les acteurs et la plateforme

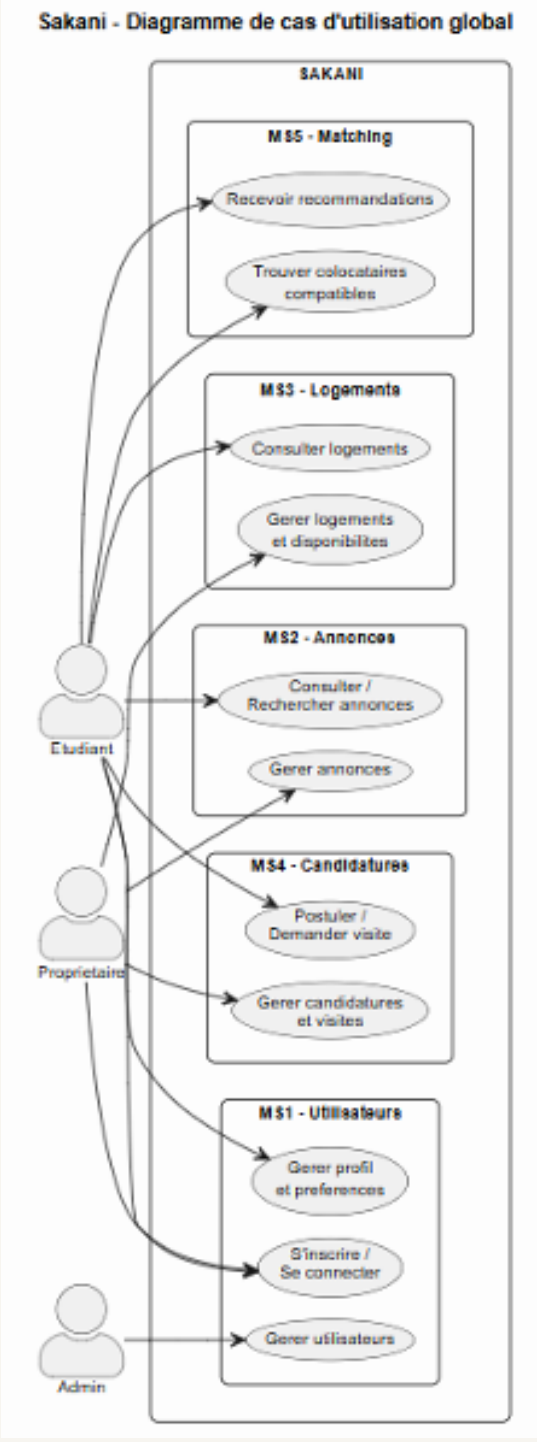


Figure 1 : Diagramme global de cas d'utilisation de SAKANI

PREMIER MICROSERVICE

User / Profile Service

Gestion des utilisateurs, des profils et des préférences

1. Objectif du microservice

Le microservice User / Profile constitue l'un des composants fondamentaux de la plateforme SAKANI. Il a pour objectif de gérer les profils des utilisateurs ainsi que leurs préférences personnelles et académiques.

Il permet de centraliser les informations relatives aux étudiants, aux propriétaires et aux administrateurs, tout en assurant la gestion de leurs profils et de leurs préférences en matière de logement et de colocation.

2. Fonctionnalités principales

Inscription et authentification

Permet aux utilisateurs de créer un compte et de se connecter à la plateforme.

Gestion des profils

Permet de créer, consulter, modifier et supprimer les profils des utilisateurs.

Gestion des rôles

Gère les différents rôles : étudiant, propriétaire et administrateur.

Préférences de logement

Permet de définir le budget, la ville et le type de logement souhaité.

Préférences de colocation

Enregistre les habitudes et les préférences personnelles des étudiants afin de faciliter la recherche de colocataires compatibles.

3. Modèle de données et entités

Le microservice repose sur trois entités principales permettant de structurer les informations des utilisateurs et leurs préférences.

Profile

Représente le profil d'un utilisateur de la plateforme.
Attributs :
Identifiant, nom, prénom, email, téléphone, université, ville et rôle.

HousingPreference

Représente les critères de recherche de logement définis par l'étudiant.
Attributs :
Budget minimum, budget maximum, ville préférée et type de logement.

LifestylePreference

Représente les habitudes et les préférences personnelles liées à la colocation.
Attributs :

Habitudes de sommeil, propreté, tabagisme, animaux et environnement d'étude.

4. Principales API REST

Les endpoints suivants permettent d'interagir avec les différentes ressources du microservice.

N°	Endpoint	Code HTTP
1	`/api/profiles` (POST)	**201**
2	`/api/profiles/{id}` (GET)	**200**
3	`/api/profiles/{id}` (PUT)	**200**
4	`/api/profiles/{id}` (DELETE)	**204**
5	`/api/profiles/{id}/housing-preferences` (PUT)	**200**
6	`/api/profiles/{id}/lifestyle-preferences` (PUT)	**200**

Remarque : Les codes HTTP indiqués correspondent aux réponses présentées dans la spécification des API du projet.

DEUXIÈME MICROSERVICE

Housing Service

Gestion des logements et des annonces immobilières

1. Objectif du microservice

Le microservice Housing Service constitue le composant responsable de la gestion des logements et de leurs annonces au sein de la plateforme SAKANI.

Il permet aux propriétaires de créer et de gérer leurs annonces immobilières, tout en offrant aux étudiants la possibilité de rechercher des logements selon différents critères et de consulter les informations détaillées des offres disponibles.

Il assure également la gestion des différents statuts des annonces, depuis leur création jusqu'à leur publication ou leur archivage.

2. Fonctionnalités principales

Fonctionnalité	Description
Création d'annonces	Permet aux propriétaires de créer des annonces pour leurs logements en ajoutant un titre et une description.
Gestion des annonces	Permet aux propriétaires de consulter, modifier et supprimer leurs annonces.
Publication des annonces	Permet de publier une annonce afin de la rendre visible aux étudiants recherchant un logement.
Recherche et filtrage	Permet aux étudiants de rechercher des annonces selon différents critères et de consulter les résultats.
Consultation des détails	Permet aux étudiants de visualiser les informations détaillées d'une annonce et du logement associé.
Gestion des statuts	Gère les différents états d'une annonce : brouillon, publiée et archivée.
Vérification des logements	Vérifie l'existence du logement associé à une annonce grâce à la communication avec le microservice Logement.

3. Modèle de données et entités

Le Housing Service repose sur trois entités principales permettant de représenter les logements, leurs équipements et leurs images.

Housing

Représente un logement proposé sur la plateforme.
Attributs :
Identifiant, propriétaire, localisation, prix et capacité.

Amenity

Représente les équipements et les commodités disponibles dans un logement.
Attribut :
Équipement.

HousingPhoto

Représente les images associées à un logement afin d'illustrer son annonce.
Attribut :
Images du logement.

4. Principales API REST

Les endpoints suivants permettent de gérer les logements et leurs annonces.

N°	Endpoint	Code HTTP
1	`POST/housings`	**201**
2	`GET/housings`	**200**
3	`GET/housings/{id}`	**200**
4	`PUT/housings/{id}`	**200**

Remarque : Les endpoints présentés correspondent aux principales opérations REST du Housing Service. Les codes HTTP indiquent le résultat attendu de chaque opération.

TROISIÈME MICROSERVICE

Application Service

Gestion du processus de candidature des étudiants

1. Objectif du microservice

Le microservice Application Service est chargé de gérer le processus de candidature des étudiants aux logements proposés sur la plateforme SAKANI.

Il permet aux étudiants de déposer leurs demandes de location, de consulter leurs candidatures et de suivre leur évolution. Il offre également aux propriétaires la possibilité de consulter les demandes reçues et de prendre une décision concernant chaque candidature.

Ce service assure aussi la traçabilité des changements de statut afin de conserver un historique des actions effectuées.

2. Fonctionnalités principales

Fonctionnalité	Description
Déposer une candidature	Permet à un étudiant d'envoyer une demande pour un logement qui l'intéresse, avec un message facultatif.
Consulter ses candidatures	Permet à l'étudiant de consulter toutes ses candidatures et leurs informations.
Consulter les détails	Affiche les informations d'une candidature : étudiant, logement, date de dépôt et statut.
Suivre le statut	Permet de suivre l'évolution d'une candidature : en attente, acceptée, refusée ou annulée.
Gérer les candidatures reçues	Permet au propriétaire de consulter les demandes reçues pour ses logements.
Accepter ou refuser	Permet au propriétaire de prendre une décision et de mettre à jour le statut d'une demande.
Annuler une candidature	Permet à l'étudiant d'annuler sa demande tant qu'elle n'a pas été traitée.
Consulter l'historique	Enregistre les changements de statut avec la date correspondante afin de garder une trace des actions.

3. Modèle de données et entités

Le microservice repose sur deux entités principales permettant de gérer les candidatures et de conserver leur historique.

RentalApplication

Représente une candidature déposée par un étudiant pour un logement.

Attributs :

`id`, `studentId`, `housingId`, `applicationDate`, `status`, `message`.

ApplicationStatusHistory

Permet de conserver l'historique des changements de statut d'une candidature.
Attributs :
`id`, `applicationId`, `oldStatus`, `newStatus`, `changedAt`.

4. Principales API REST

Les endpoints suivants permettent de consulter, de suivre et de gérer les candidatures.

N°	Endpoint	HTTP
1	`GET/applications/{id}`	**200 OK**
2	`GET/applications?studentId={id}`	**200 OK**
3	`PUT/applications/{id}/status`	**200 OK**
4	`DELETE/applications/{id}`	**204 No Content**

Remarque : Les méthodes HTTP sont indiquées dans la présentation du projet. Les codes de réponse HTTP ne sont pas spécifiés dans celle-ci.

QUATRIÈME MICROSERVICE

Visit Service

Planification et gestion des visites de logements

1. Objectif du microservice

Le microservice Visit Service a pour objectif de gérer et d'organiser les visites des logements étudiants au sein de la plateforme SAKANI. Il facilite la coordination entre les étudiants et les propriétaires en centralisant la gestion des créneaux de visite et des demandes de réservation.

Il permet aux propriétaires de proposer des disponibilités pour leurs logements, tandis que les étudiants peuvent consulter les créneaux disponibles et demander une visite.

Le service prend également en charge la confirmation, le refus et l'annulation des réservations, tout en garantissant qu'un même créneau ne soit pas réservé par plusieurs étudiants.

2. Fonctionnalités principales

Fonctionnalité	Description
Création des créneaux de visite	Permet aux propriétaires de proposer des dates et des horaires pour visiter leurs logements.
Modification et annulation des créneaux	Permet aux propriétaires de modifier ou de supprimer les créneaux proposés.
Consultation des disponibilités	Permet aux étudiants de consulter les créneaux disponibles pour un logement.
Demande de réservation	Permet aux étudiants de demander une visite en choisissant un créneau disponible.
Confirmation ou refus des demandes	Permet aux propriétaires d'accepter ou de refuser les demandes de visite.
Gestion et suivi des visites	Permet de suivre le statut des visites : en attente, confirmée, refusée ou annulée.
Prévention des doubles réservations	Empêche plusieurs étudiants de réserver le même créneau de visite.

3. Modèle de données et entités

Le Visit Service repose sur deux entités principales qui permettent de gérer les disponibilités et les réservations de visites.

VisitSlot

Représente un créneau de visite proposé par un propriétaire pour permettre aux étudiants de visiter un logement.

Rôle principal :

Gérer les dates et les horaires de disponibilité des logements pour les visites.

VisitBooking

Représente une demande de réservation de visite effectuée par un étudiant.

Rôle principal :

Gérer le cycle de vie d'une réservation, depuis la demande initiale jusqu'à sa confirmation, son refus ou son annulation.

4. Principales API REST

Les endpoints suivants permettent de gérer les créneaux de visite et les réservations des étudiants.

N°	Endpoint	Code HTTP
1	`POST/visit-slots`	**201**
2	`GET/visit-slots`	**200**
3	`PUT/visit-slots/{id}`	**200**
4	`DELETE/visit-slots/{id}`	**204**
5	`POST/visit-bookings`	**201**
6	`PUT/visit-bookings/{id}/status`	**200**
7	`DELETE/visit-bookings/{id}`	**204**

Remarque : Les codes HTTP proposés correspondent aux réponses conventionnelles des opérations REST. Les chemins présentés constituent une proposition de structuration des endpoints et doivent être vérifiés avec l'implémentation réelle du service.

CINQUIÈME MICROSERVICE

Intelligent Matching Service

Matching intelligent et recommandations personnalisées

1. Objectif du microservice

Le microservice Intelligent Matching Service a pour objectif de proposer aux étudiants des logements et des colocataires compatibles avec leurs besoins et leurs préférences.

Il exploite les informations des profils étudiants, notamment leurs préférences de logement et leurs habitudes de vie, afin de calculer des scores de compatibilité et de générer des recommandations personnalisées.

Ce service permet ainsi de faciliter la recherche de colocataires partageant des critères similaires et de classer les logements selon leur pertinence pour chaque étudiant.

2. Fonctionnalités principales

Fonctionnalité	Description
Matching des colocataires	Permet de trouver des colocataires compatibles selon les préférences et les habitudes des étudiants.
Calcul de compatibilité	Calcule un score de compatibilité entre deux étudiants selon leurs profils et leurs préférences de colocation.
Recommandation de logements	Propose des logements adaptés au budget, à la localisation et aux préférences de chaque étudiant.
Recommandation de colocataires	Suggère des colocataires potentiels ayant des habitudes et des critères similaires.
Classement des recommandations	Classe les logements et les colocataires selon leur score de compatibilité.
Personnalisation des résultats	Adapte les recommandations en fonction des préférences enregistrées dans le profil de l'étudiant.
Mise à jour des recommandations	Actualise les suggestions lorsque les préférences des étudiants ou les informations des logements changent.

3. Modèle de données et entités

Le Matching Service repose sur trois entités principales permettant de gérer les critères de recherche, les scores de compatibilité et les recommandations générées.

MatchingCriteria

Représente les critères utilisés pour comparer les profils des étudiants et identifier les préférences communes.

CompatibilityScore

Représente le score calculé pour mesurer le niveau de compatibilité entre deux étudiants ou entre un étudiant et une recommandation.

Recommendation

Représente une suggestion personnalisée de logement ou de colocataire proposée à un étudiant selon ses critères et son score de compatibilité.

4. Principales API REST

Les endpoints suivants permettent de rechercher des colocataires compatibles et de proposer des logements adaptés aux étudiants.

N°	Endpoint	Code HTTP
1	`GET/matches/roommates/{studentId}`	**200**
2	`GET/recommendations/housings/{studentId}`	**200**

Remarque : Les deux endpoints sont repris de la présentation du projet. Le code HTTP 200 correspond à une réponse réussie conventionnelle pour ces opérations de consultation.

MICROSERVICE 01

Diagramme de classes

User / Profile Service

1. Représentation du diagramme

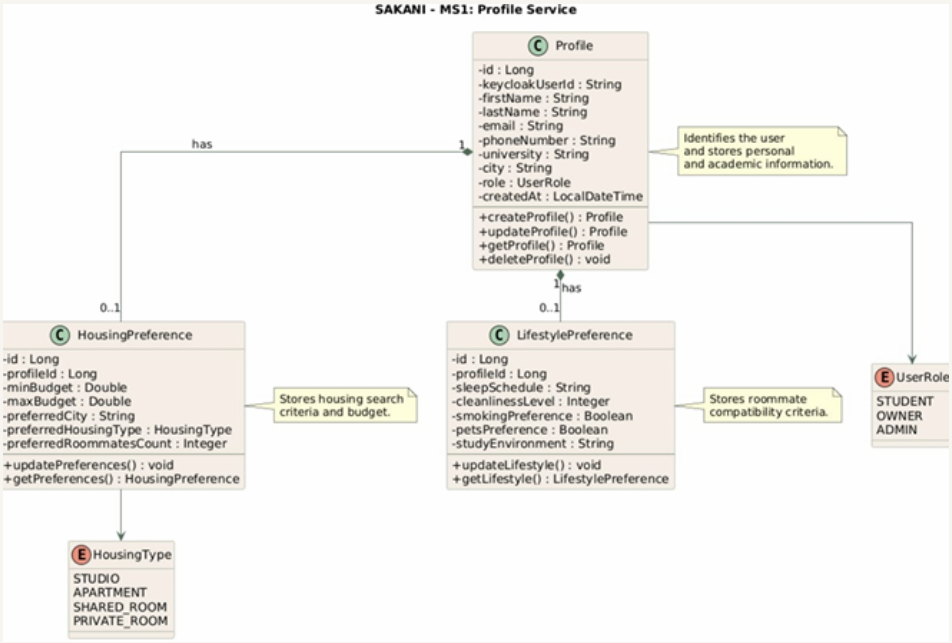


Figure : Diagramme de classes du User / Profile Service

2. Description des entités

2.1 Profile

L'entité **Profile** représente le profil personnel d'un utilisateur inscrit sur la plateforme SAKANI. Elle centralise ses informations d'identification et ses informations générales, notamment son nom, son prénom, son adresse email, son numéro de téléphone, son université, sa ville et son rôle au sein de la plateforme.

2.2 HousingPreference

L'entité **HousingPreference** regroupe les préférences de logement définies par l'étudiant. Elle permet d'enregistrer son budget minimum, son budget maximum, la ville souhaitée ainsi que le type de logement recherché. Ces informations facilitent la recherche de logements correspondant aux besoins de chaque étudiant.

2.3 LifestylePreference

L'entité **LifestylePreference** représente les habitudes de vie et les préférences personnelles de l'étudiant. Elle prend en compte des critères tels que les habitudes de sommeil, la propreté, le tabagisme, la présence d'animaux et l'environnement d'étude. Ces informations contribuent au fonctionnement du système de matching et à la recherche de colocataires compatibles.

MICROSERVICE 02

Diagramme de classes

Housing Service

1. Représentation du diagramme

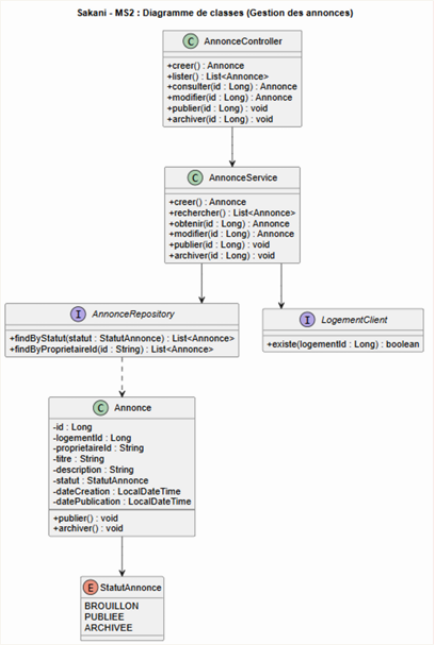


Figure : Diagramme de classes du Housing Service

2. Description des entités

2.1 Housing

L'entité **Housing** représente un logement publié sur la plateforme SAKANI par un propriétaire. Elle constitue l'entité principale du microservice et centralise les informations relatives au logement, notamment sa description, sa localisation, son prix, sa capacité d'accueil et ses caractéristiques. Elle permet aux étudiants de consulter les offres disponibles et de rechercher un logement adapté à leurs besoins.

2.2 Amenity

L'entité **Amenity** représente les équipements et les commodités disponibles dans un logement. Elle permet de décrire les différents services et installations proposés, tels que le Wi-Fi, la climatisation, le chauffage, le parking ou encore les équipements électroménagers. Elle aide les étudiants à identifier les logements correspondant à leurs exigences.

2.3 HousingPhoto

L'entité **HousingPhoto** permet de gérer les photographies associées à chaque logement. Elle facilite la présentation visuelle des annonces en permettant de stocker les références des images et de les associer aux logements correspondants. Ces photographies offrent aux étudiants une meilleure visibilité sur l'état, l'aménagement et les différentes caractéristiques des logements proposés.

MICROSERVICE 03

Diagramme de classes

Application Service

1. Représentation du diagramme

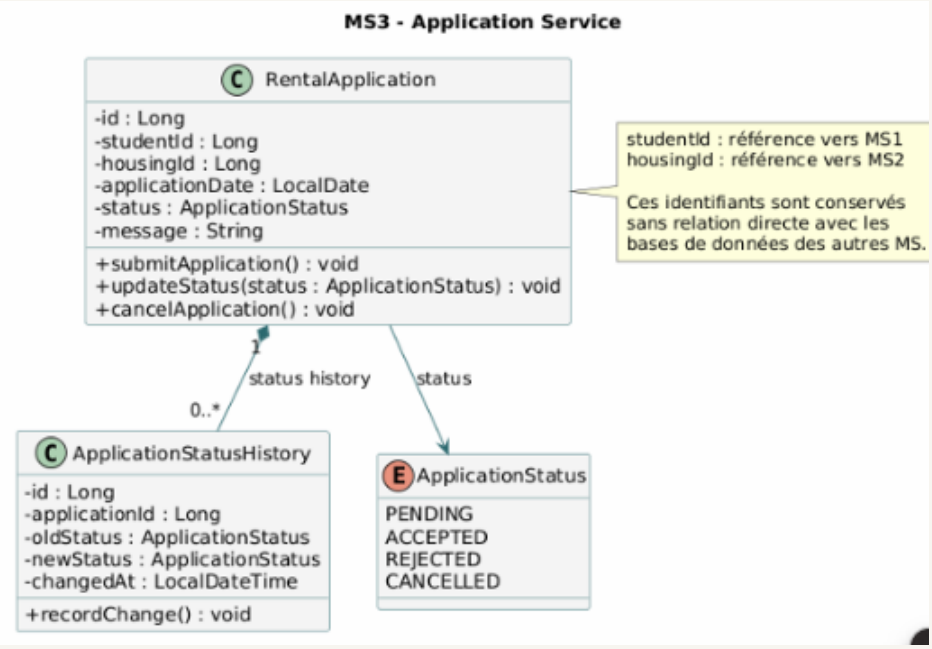


Figure : Diagramme de classes de l'Application Service

2. Description des entités

2.1 RentalApplication

L'entité **RentalApplication** représente une candidature déposée par un étudiant pour un logement qui l'intéresse. Elle constitue l'entité principale du microservice et permet d'enregistrer les informations relatives à chaque demande de location.

Elle contient les attributs suivants : **id**, **studentId**, **housingId**, **applicationDate**, **status** et **message**. Ces informations permettent d'identifier l'étudiant concerné, le logement demandé, la date de dépôt ainsi que l'état d'avancement de la candidature.

2.2 ApplicationStatusHistory

L'entité **ApplicationStatusHistory** permet de conserver l'historique des changements de statut d'une candidature. Elle assure la traçabilité des différentes modifications effectuées au cours du traitement d'une demande.

Elle contient les attributs suivants : **id**, **applicationId**, **oldStatus**, **newStatus** et **changedAt**. Ces informations permettent d'identifier la candidature concernée, de connaître son ancien et son nouveau statut, ainsi que la date à laquelle la modification a été effectuée.

MICROSERVICE 04

Diagramme de classes

Visit Service

1. Représentation du diagramme

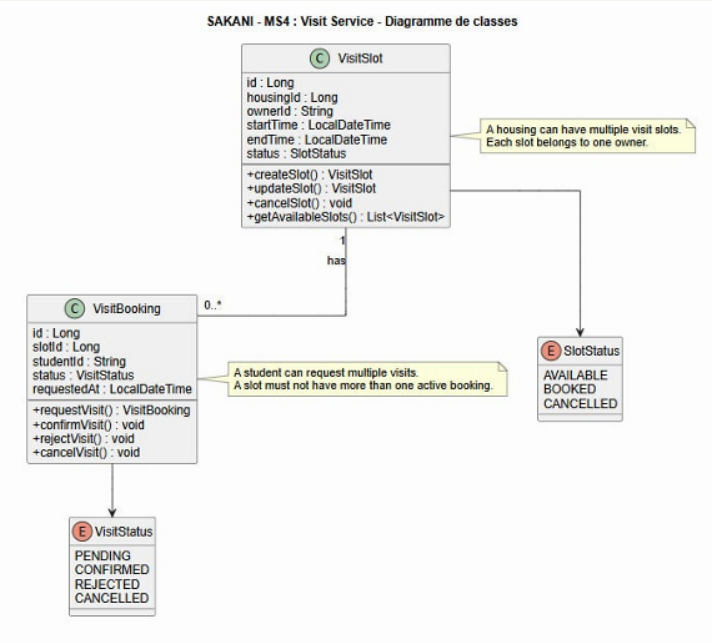


Figure : Diagramme de classes du Visit Service

2. Description des entités

2.1 VisitSlot

L'entité **VisitSlot** représente un créneau horaire proposé par un propriétaire pour permettre aux étudiants de visiter un logement. Elle permet de définir les disponibilités de visite en précisant la date, l'heure et les informations associées au logement concerné.

Elle facilite ainsi l'organisation des visites et permet aux étudiants de consulter les créneaux disponibles avant de soumettre une réservation.

2.2 VisitBooking

L'entité **VisitBooking** représente une réservation de visite effectuée par un étudiant pour un créneau disponible. Elle permet d'enregistrer les informations relatives à la demande de visite, notamment l'étudiant concerné, le créneau sélectionné et le statut de la réservation.

Elle assure le suivi des réservations et facilite la coordination entre les étudiants et les propriétaires, depuis la demande de visite jusqu'à sa confirmation ou son refus.

MICROSERVICE 05

Diagramme de classes

Intelligent Matching Service

1. Représentation du diagramme

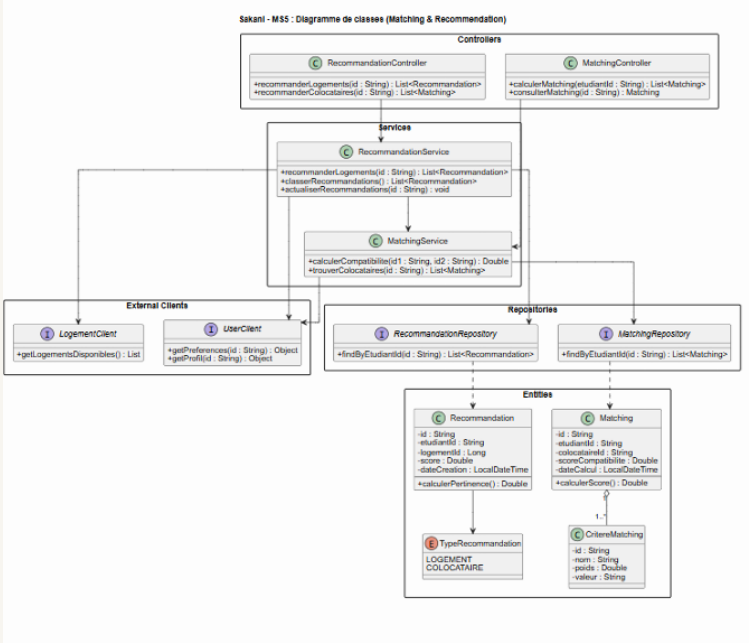


Figure : Diagramme de classes de l'Intelligent Matching Service

2. Description des entités

2.1 MatchingCriteria

L'entité **MatchingCriteria** représente les critères utilisés par le système pour évaluer la compatibilité entre les étudiants. Elle regroupe les préférences et les caractéristiques prises en compte lors du processus de matching, telles que les habitudes de vie, les préférences personnelles et les critères de colocation.

Ces informations permettent au système de comparer les profils des étudiants afin d'identifier des colocataires potentiellement compatibles.

2.2 CompatibilityScore

L'entité **CompatibilityScore** représente le résultat de l'évaluation de compatibilité effectuée par le système. Elle permet d'attribuer un score aux profils comparés afin de mesurer leur niveau de correspondance selon les critères définis.

Ce score constitue un élément essentiel du processus de matching, puisqu'il permet de classer les profils et de mettre en avant les étudiants présentant les meilleures compatibilités.

2.3 Recommendation

L'entité **Recommendation** représente les suggestions générées par le système à destination des étudiants. Elle permet de proposer des logements ou des colocataires correspondant aux préférences et aux critères de recherche de chaque utilisateur.

Les recommandations peuvent être organisées selon leur pertinence et leur score de compatibilité, afin d'aider les étudiants à prendre des décisions plus adaptées à leurs besoins.

MICROSERVICE 01

Communication inter-microservices

User / Profile Service

Modes de communication synchrone et asynchrone

Le microservice **User / Profile Service** est chargé de gérer les profils des utilisateurs ainsi que leurs préférences personnelles et académiques. Dans l'architecture SAKANI, sa communication avec les autres microservices permet de partager les informations nécessaires au fonctionnement des différentes fonctionnalités de la plateforme.

1. Communication synchrone

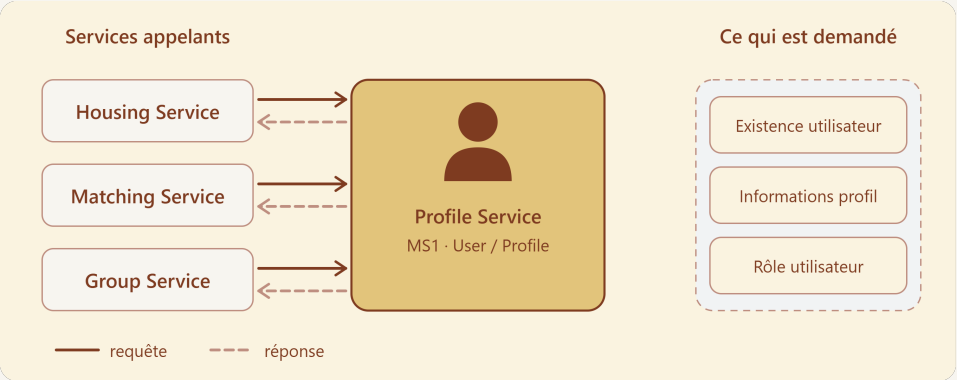


Figure : Communication synchrone du User / Profile Service

La communication synchrone repose sur un échange direct entre deux microservices. Le service demandeur envoie une requête et attend la réponse du service sollicité avant de poursuivre son traitement. Ce mécanisme permet d'obtenir immédiatement les informations nécessaires à une opération.

2. Communication asynchrone

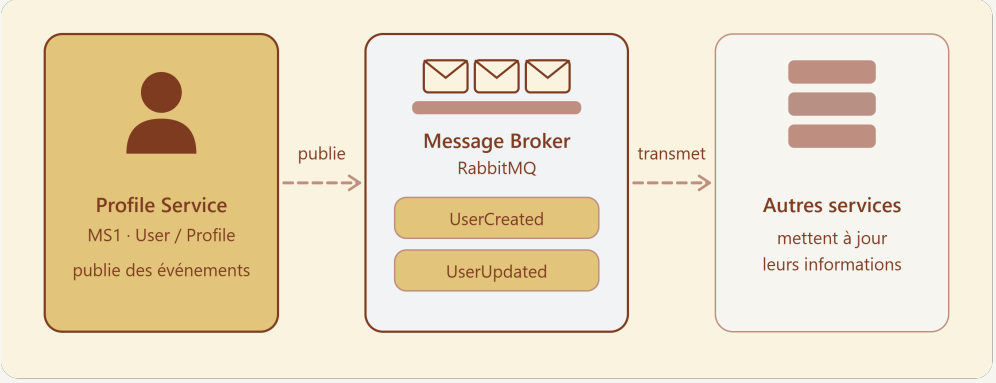


Figure : Communication asynchrone du User / Profile Service

La communication asynchrone permet aux microservices d'échanger des informations sans attendre directement une réponse immédiate. Le service émetteur transmet un événement ou un message qui pourra être traité ultérieurement par le service destinataire. Cette approche favorise le découplage et l'indépendance des traitements entre les différents services.

MICROSERVICE 02

Communication inter-microservices

Housing Service

Modes de communication synchrone et asynchrone

Le microservice **Housing Service** est chargé de la gestion des logements et de leurs informations sur la plateforme SAKANI. Il permet notamment de créer, consulter et mettre à jour les annonces immobilières. Sa communication avec les autres microservices facilite l'accès aux informations des logements et la coordination des différentes opérations liées à leur location.

1. Communication synchrone

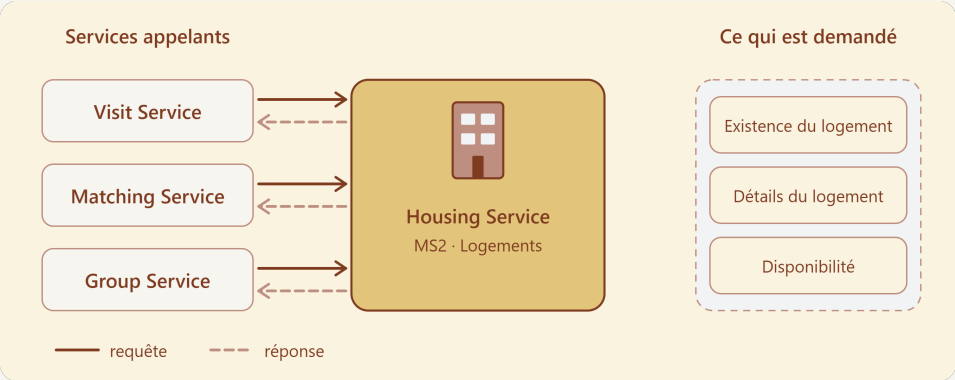


Figure : Communication synchrone du Housing Service

La communication synchrone permet au Housing Service d'échanger directement avec les autres microservices lorsqu'une opération nécessite une réponse immédiate. Le service demandeur envoie une requête pour obtenir les informations relatives à un logement ou vérifier certaines données nécessaires au traitement de sa demande. Le traitement se poursuit après réception de la réponse.

2. Communication asynchrone

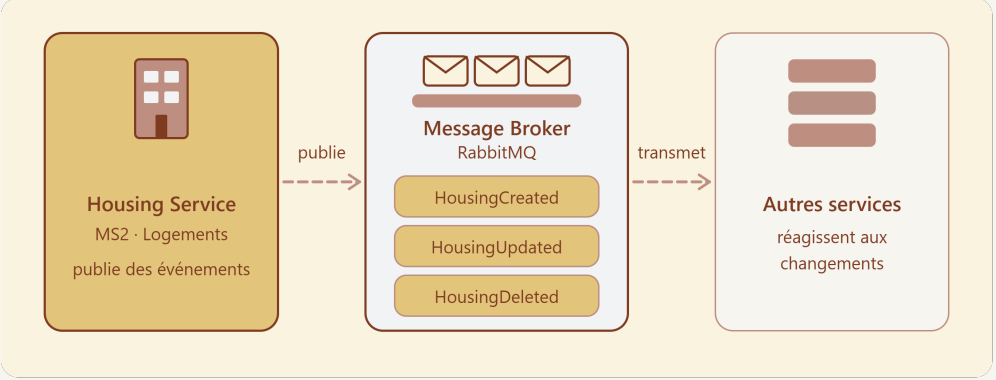


Figure : Communication asynchrone du Housing Service

La communication asynchrone permet au Housing Service de transmettre des événements liés aux logements sans attendre le traitement immédiat des services destinataires. Les messages peuvent être consommés ultérieurement, ce qui favorise le découplage des traitements et permet aux différents services de fonctionner de manière plus indépendante.

MICROSERVICE 03

Communication inter-microservices

Application Service

Modes de communication synchrone et asynchrone

Le microservice **Application Service** assure la gestion des candidatures déposées par les étudiants pour les logements proposés sur la plateforme SAKANI. Il permet d'enregistrer les demandes de location, de suivre leur état d'avancement et de conserver l'historique des changements de statut. Sa communication avec les autres microservices contribue à coordonner le processus de candidature et le traitement des demandes.

1. Communication synchrone

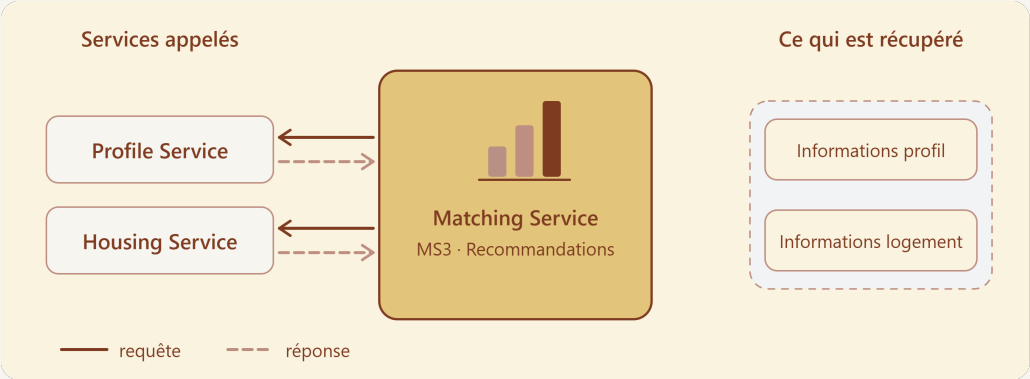


Figure : Communication synchrone de l'Application Service

La communication synchrone permet à l'Application Service de solliciter directement un autre microservice lorsqu'il a besoin d'informations pour traiter une candidature. Il envoie une requête et attend la réponse du service destinataire avant de poursuivre son traitement. Ce mécanisme permet notamment de récupérer les données nécessaires à la vérification et au suivi des demandes de location.

2. Communication asynchrone

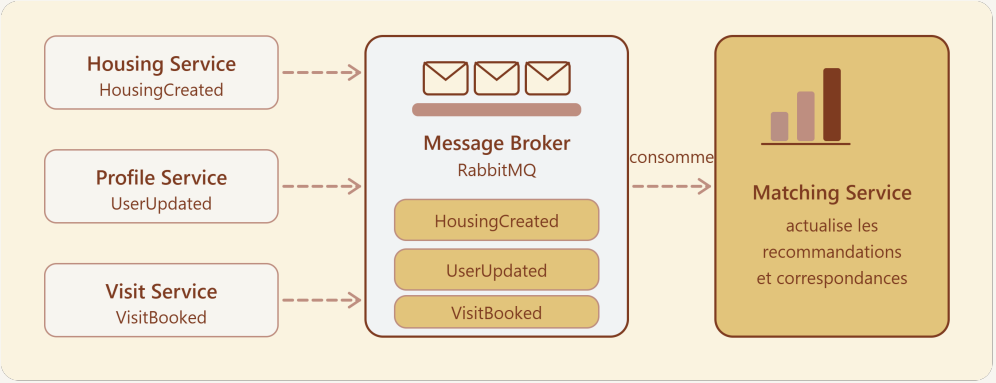


Figure : Communication asynchrone de l'Application Service

La communication asynchrone permet à l'Application Service de publier des événements liés aux candidatures, comme la création d'une demande ou la modification de son statut. Les autres microservices concernés peuvent recevoir et traiter ces événements sans bloquer le service émetteur. Cette approche facilite la propagation des changements et améliore l'indépendance des traitements au sein de l'architecture.

MICROSERVICE 04

Communication inter-microservices

Visit Service

Modes de communication synchrone et asynchrone

Le microservice **Visit Service** est responsable de la gestion et de l'organisation des visites des logements étudiants sur la plateforme SAKANI. Il permet aux propriétaires de proposer des créneaux de visite et aux étudiants de consulter les disponibilités et de réserver un rendez-vous. Il assure également le suivi des réservations, leur confirmation ou leur refus, ainsi que la prévention des doubles réservations.

1. Communication synchrone

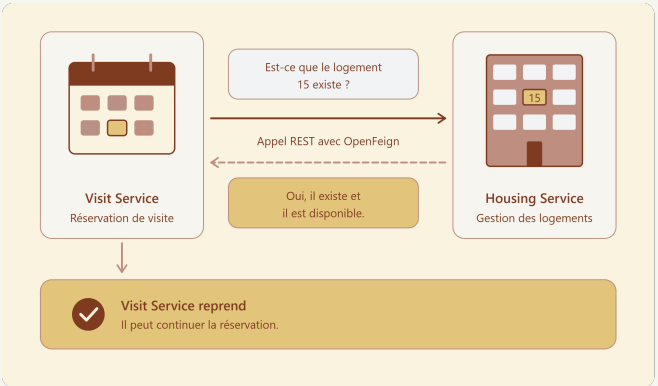


Figure : Communication synchrone du Visit Service

La communication synchrone permet au Visit Service d'interroger directement les autres microservices lorsqu'il a besoin de données pour gérer une visite. Par exemple, il peut vérifier l'existence d'un logement auprès du Housing Service avant de traiter une demande de réservation. Le service attend la réponse avant de poursuivre l'opération, garantissant ainsi la cohérence des informations utilisées.

2. Communication asynchrone

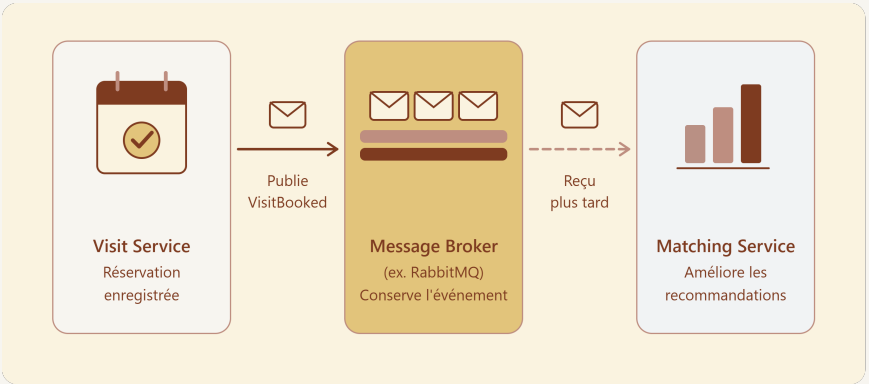


Figure : Communication asynchrone du Visit Service

La communication asynchrone permet au Visit Service de diffuser des événements liés aux visites, notamment la création d'une réservation, sa confirmation, son refus ou son annulation. Les services destinataires peuvent recevoir et traiter ces événements indépendamment du service émetteur. Ce mécanisme facilite la coordination des traitements et réduit le couplage entre les différents microservices.

MICROSERVICE 05

Communication inter-microservices

Intelligent Matching Service

Modes de communication synchrone et asynchrone

Le microservice **Intelligent Matching Service** est chargé d'identifier les compatibilités entre les étudiants afin de faciliter la recherche de colocataires. Il exploite les informations des profils et les préférences de vie pour calculer des scores de compatibilité et générer des recommandations personnalisées. Sa communication avec les autres microservices lui permet de récupérer les données nécessaires au processus de matching et de diffuser les résultats obtenus.

1. Communication synchrone

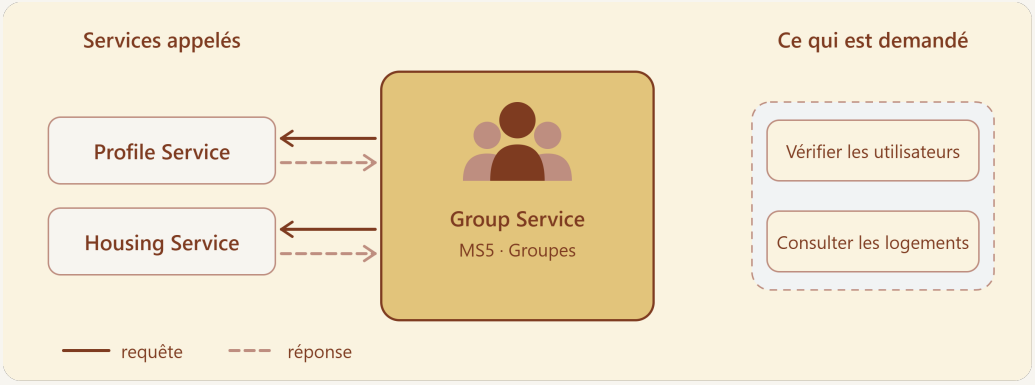


Figure : Communication synchrone de l'Intelligent Matching Service

La communication synchrone permet à l'Intelligent Matching Service de récupérer directement les informations nécessaires auprès des autres microservices, notamment les profils des étudiants et leurs préférences de colocation. Le service attend les réponses avant de procéder à l'analyse

des données et au calcul des scores de compatibilité. Ce mécanisme garantit la disponibilité des informations requises pour produire des recommandations pertinentes.

2. Communication asynchrone

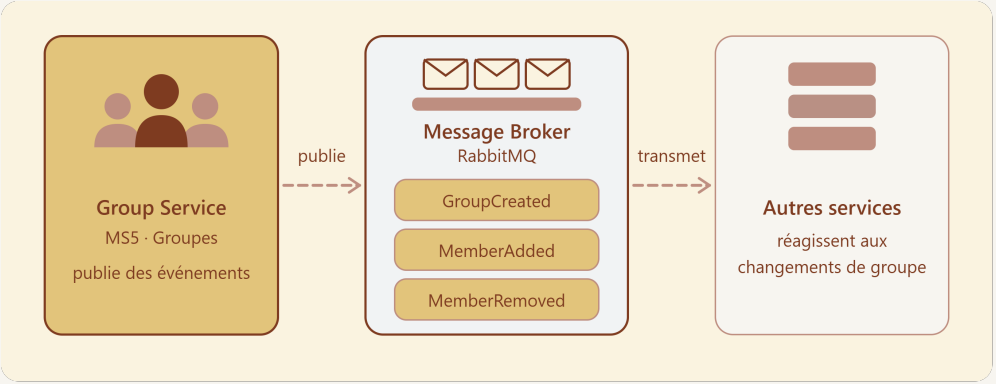


Figure : Communication asynchrone de l'Intelligent Matching Service

La communication asynchrone permet à l'Intelligent Matching Service de recevoir ou de diffuser des événements liés aux profils et aux préférences des étudiants, ainsi que les résultats du matching. Les messages sont transmis sans exiger une réponse immédiate du service destinataire. Cette approche permet de déclencher les traitements de matching indépendamment des autres opérations et de diffuser les recommandations aux services concernés.

BILAN GÉNÉRAL

Conclusion générale

Le projet **SAKANI** a pour objectif de répondre aux difficultés rencontrées par les étudiants lors de la recherche d'un logement adapté à leurs besoins. À travers une plateforme web centralisée, il propose une solution permettant de faciliter les interactions entre les étudiants et les propriétaires, tout en simplifiant les différentes démarches liées à la location et à la colocation.

La conception de cette plateforme repose sur une architecture en microservices composée de cinq services indépendants, chacun étant chargé d'une responsabilité spécifique. Cette organisation permet de structurer les différentes fonctionnalités de l'application, de faciliter les échanges entre les services et d'assurer une meilleure modularité de l'ensemble du système.

Au cours de cette étape du projet, nous avons étudié les besoins fonctionnels, identifié les différents acteurs, défini les responsabilités des microservices et élaboré les diagrammes nécessaires à leur conception. Nous avons également présenté les mécanismes de communication synchrones et asynchrones permettant aux différents services de collaborer.

Cette première phase constitue une base importante pour la suite du développement de SAKANI. Elle permet de disposer d'une vision structurée de la solution et prépare les prochaines étapes, notamment l'implémentation, l'intégration et la validation des différentes fonctionnalités.

Ainsi, SAKANI ambitionne de rendre la recherche de logement étudiant plus simple, plus organisée et plus adaptée aux besoins de chaque utilisateur.

SAKANI

Find your place. Find your people.